

SDE I Take-Home Assignment: Support Inbox AI Triage

Candidate Assessment | Confidential

Position	SDE I
Duration	12-24 hours
Stack	Node.js / TypeScript, React preferred, SQLite/JSON/in-memory storage, LLM provider or mock mode
Confidential	Candidate assessment

Build a support inbox where incoming tickets are classified, prioritized, and given a suggested response for human review.

Overview

Support teams need to identify urgent issues quickly and respond consistently. In this assignment, you will build an AI-assisted support inbox that helps triage customer messages.

The system should classify tickets, suggest priority and category, draft a response, and let a human accept, edit, or reject the AI suggestion.

The emphasis is on a realistic review workflow, not fully automated customer communication.

What We Are Looking For

- Practical full-stack implementation with clean ticket models and status transitions.
- Structured LLM output for classification and draft generation.
- A review UI that keeps a human in control.
- Basic analytics and clear error/loading states.

What to Deliver

1. A GitHub repository or zip archive with complete source code and setup instructions.
2. A README.md covering setup, architecture, AI classification approach, assumptions, and limitations.
3. A 3-6 minute Loom or screen recording showing ticket creation, triage, review, and analytics.
4. Seed tickets representing different categories and urgency levels.

Important

- The system should not send real emails. Draft responses should remain inside the app.
- Mock LLM mode is acceptable if the integration boundary is clear.
- Keep categories and priorities simple and documented.

Part 1: Core Project

1A. Backend API and Data Model

Create a REST API for tickets, triage results, draft responses, and human review state.

- Tickets: create/list/read/update with customer name, email, subject, message, status, created_at, and updated_at.
- Triage result: category, sentiment, urgency, priority, confidence, and explanation.
- Draft response: suggested reply text, tone, status, and reviewer edits.
- Review workflow: pending_review, accepted, edited, rejected, resolved.
- Endpoints to run or re-run triage for a ticket.
- Consistent validation and error handling.

1B. AI Triage and Drafting

Use an LLM or mock service to analyze ticket content and produce structured results.

- Classify ticket category, such as billing, bug, feature request, account access, or general question.
- Detect urgency and sentiment.
- Suggest a priority level with a short rationale.
- Generate a draft support response that is professional and concise.
- Return structured JSON and validate it before storage.
- Handle low-confidence or invalid outputs gracefully.

1C. Frontend Interface

Build an inbox-style UI that makes triage and review easy.

- Ticket inbox with filters for status, category, and priority.
- Ticket detail view showing original message, AI triage result, explanation, and confidence.
- Draft response editor where a reviewer can accept, edit, or reject the suggested response.
- Status transitions visible in the UI.
- Basic analytics view showing ticket counts by category, priority, and status.
- Loading and error states for triage actions.

1D. Tests and Documentation

Add simple tests for structured AI parsing and review transitions.

- Unit test for triage output schema validation.
- Unit or integration test for accept/edit/reject flow.
- README with setup steps, data model overview, assumptions, and known limitations.
- Document what would be required before using this in a real support system.

Part 2: Bonus Extensions (Pick Any)

The bonus extensions are not required. Partial implementations with clear documentation are better than broad, unstable work. Pick whichever extension best matches your strengths.

Extension A: Bulk Triage

Requirements

5. Select multiple untriaged tickets and run triage in bulk.
6. Show progress and per-ticket errors.
7. Avoid blocking the entire batch because one ticket fails.

What this tests: Tests simple async workflows and error handling.

Extension B: SLA Warning

Requirements

8. Add a configurable SLA threshold for urgent tickets.
9. Highlight urgent tickets that have been pending too long.

10. Add a simple dashboard count for SLA-risk tickets.

What this tests: Tests business rules and dashboard thinking.

Extension C: Similar Ticket Suggestions

Requirements

11. Show similar past tickets based on category or keyword match.

12. Display previous resolution notes if available.

13. Explain your matching approach in the README.

What this tests: Tests lightweight retrieval and product usefulness.

Evaluation Criteria

Core Project (80%)

Criteria	Weight	Description
Backend and Data Model	20%	Ticket models, triage result storage, review workflow, validation, and API consistency.
AI Integration	20%	Classification quality, structured output parsing, draft generation, and error handling.
Frontend Flow	20%	Inbox, ticket detail, review editor, filters, and analytics usability.
End-to-End Functionality	15%	Can reviewers create tickets, run triage, review drafts, and update status?
Code Quality	5%	TypeScript usage, readability, tests for core flows, and maintainability.

Bonus Extensions (20%)

Criteria	Weight	Description
Completeness	8%	Selected extension works or is clearly documented as partial.
Product Thinking	7%	Extension makes the support workflow more realistic.
Trade-off Awareness	5%	README explains skipped production features and risks.

Guidelines and Constraints

Required Stack

- Backend: Node.js with TypeScript.
- Frontend: React with TypeScript preferred.
- LLM: provider of choice or documented mock mode.
- Database: SQLite, JSON file, or in-memory storage.
- Validation: Zod or equivalent preferred.

Out of Scope

- Sending real customer emails.
- Authentication, agent assignment, and full CRM integration.
- Complex analytics or reporting infrastructure.
- Production deployment.

Specifically Encouraged

- Seed tickets with clear expected triage behavior.
- Editable AI drafts and clear human review actions.
- Simple status transition logic.
- README discussion of AI confidence and human oversight.

Suggested Time Allocation

Phase	Time	Notes
Plan ticket states and schema	30-60 min	Define categories and statuses early.
Backend API	3-4 hrs	Ticket CRUD and review states first.
AI triage service	2-4 hrs	Use structured JSON and validation.
Frontend	4-6 hrs	Inbox and detail review are most important.
Tests and README	1-2 hrs	Cover parsing and review transitions.
Bonus extension	2-4 hrs	Bulk triage or SLA warning are good fits.

Questions?

If anything in this assignment is ambiguous, make a reasonable assumption and document it in your README. The goal is to see how you decompose scope, communicate trade-offs, and build a reliable product loop under time constraints.